SHL Assessment Recommendation System

Approach Document | Author: Mohammad Inayat Hussain

Problem, Architecture & Data Pipeline

Problem Statement

Given a natural-language job description or hiring query, recommend the most relevant SHL assessments from the product catalog. The system must expose a REST API (GET and POST), return results in a specific JSON format, and produce a CSV of predictions for batch evaluation.

Architecture

The system follows a straightforward retrieval pipeline:

```
User Query -> Sanitize -> Extract Constraints -> Expand (Gemini LLM)
  -> Embed (Sentence-Transformer) -> Hybrid Score -> Filter & Rank -> Top 10
```

Three-stage scoring:

* Semantic similarity (50%) ? Cosine similarity between the query embedding and precomputed assessment embeddings using all-MiniLM-L6-v2.

* Keyword overl
catches exact tech

Constraint extraction parses the query for:

* Time limits

Data Pipeline

1. Scraping ? Built a requests + BeautifulSoup scraper that paginates through SHL's product catalog (type=1 for Individual Test Solutions). Visited each detail page to extract descriptions, test types, durations, remote/adaptive support, and job levels. Handled SHL's occasional 5xx errors with retry logic and polite delays.

2. Cleaning ? The raw scraper output needed fixes: test types were space-separated on SHL's site ("C P A B"), durations had missing values (set to -1 as sentinel), and some entries had unicode dashes that broke matching. A cleaning script handles column renaming, deduplication, URL validation, and builds a combined_text column for embeddings.

3. Final catalog: 389 Individual Test Solutions with structured metadata.

Tech Stack

Layer	Technology
API	FastAPI + Uvicorn
Embeddings	sentence-transformers/all-MiniLM-L6-v2 (384-dim)
LLM	Google Gemini 2.0 Flash (query expansion, optional)
Scoring	NumPy + scikit-learn (cosine similarity)
Frontend	Streamlit
Scraping	requests + BeautifulSoup4
Deployment	Hugging Face Spaces (API) + Streamlit Cloud (frontend)

Evaluation, Decisions & Security

Evaluation Journey

The primary metric is Mean Recall@10 against the labeled training set (10 queries, each with 1-3 ground truth assessments).

Iteration	What Changed	Mean Recall@10
1	Baseline semantic search only	0.567
2	Added fuzzy name matching (initially broken by unicode)	0.450
3	Fixed name resolution + manual override map	0.600
4	Synonym expansion + adaptive boost + name-match scoring	0.633

Key improvements in the final iteration:

* Domain-spe
gap between h

Design Decisions

* Why hybrid instead of pure semantic? Pure embedding search missed exact technology names. A query for "Java 8" would return general coding tests because the embedding space does not differentiate well between specific tech stacks. Adding keyword overlap and name matching solved this.

* Why Gemini is
pipeline never fails

Security Measures

These are implemented and active, not theoretical:

* Input saniti
10,000 charac

Production upgrade path: JWT/OAuth2 authentication, CORS restricted to known frontend domains, structured audit logging.

Known Limitations

* LinkedIn URL scraping ? LinkedIn blocks automated requests. Pasted LinkedIn URLs fall back to slug-based text extraction, which is less accurate than the full job description.

* Query ambiguit
relevance drops.

Future Work

* Fine-tune embeddings on SHL-specific assessment-query pairs

* Add re-ranking